

14/7/26

DATE

## Python Programming Assignment - 25

3

Ans

1) What is Object - Oriented Programming?

Ans: Object Oriented Programming (OOP) is a programming paradigm built around the concept of "objects". Instead of just writing a sequence of functions & logic (procedural programming), OOP bundles data (attribute/variables) and behaviours (methods/functions) together into reusable, structured objects. It helps software model real-world entities and relationships.

2) What are the four core characteristics of OOP?

Ans: The four fundamental pillars of OOP are:

- i) Encapsulation: Bundling data and the methods that operate on that data into a single unit (class), and restricting direct access to some of the object's components.
- ii) Abstraction: Hiding complex implementation details and showing only the essential features of an object.
- iii) Inheritance: A mechanism where a new class derives properties and behaviours from an existing class, promoting code reusability.
- iv) Polymorphism: The ability of different objects to respond to the same method call in their own specific way (eg, a speak() method doing different things for a Dog class versus a Cat class).



3) Explain class and object with real-world analogy.

Ans:

- Class: A blueprint, template, or design. It defines what properties and behaviours an entity should have, but it doesn't actually exist as a physical thing yet.
- Object: An actual, usable instance created from that blueprint.

Real-world analogy:

Think of a car blueprint (the class). The blueprint dictates that a car will have attributes (color, brand, engine type) and behaviours (start, stop, accelerate). An Object is a specific, physical car built from that blueprint - like your red Toyota parked in the driveway. You can have many objects (cars) built from the same single class (blueprint), each with different colors or brands.

4) What is a constructor in Python? Why is `--init--()` used?

Ans:

A constructor is a special magic method that is automatically called the moment a new object is created from a class.

In Python, this method is specifically named `--init--()` (short for initialize). It is used to set up the initial state of the object by assigning values to its instance variables as soon as the object comes into existence.



5) Is constructor mandatory in a class? Explain.

Ans: No, it is not mandatory to write one. If you do not explicitly define an `--init--()` method in your class, Python will automatically provide a default, empty constructor behind the scenes. However, if your object needs to start with specific data or state (like a bank account starting with a specific balance), you must explicitly write one.

6) What is a destructor? When is `--del--()` called?

Ans: A destructor is the opposite of a constructor. It is a special method called `--del--()` that gets triggered automatically when an object is about to be destroyed or deleted from memory.

It is called when the object's reference count reaches zero, meaning the program is no longer using the object, and Python's garbage collector steps in to free up the memory. It is typically used for clean-up actions like closing files or network connections.

7) Difference between constructor and destructor.

Ans:

**Constructor:**

- Purpose: Initialises a newly created object.
- Timing: Called automatically when the object is created.
- Common Use: Setting up instance variables.

**Destructor:**

- Purpose: Cleans up resources before an object is destroyed.
- Timing: Called automatically before the object is destroyed.
- Common Use: Closing open files, databases, or network connections.



8) What are instance variables and class variables?

Ans: • **Instance Variables:**

Variables attached to a specific object (instance). They are usually defined inside the `--init--()` method using `self` (eg, `self.name = "John"`). Each object gets its own separate copy of these variables.

• **Class Variables:**

Variables shared by all objects of a class. They are defined inside the class structure but outside any specific method. If one object changes a class variable, it changes for all objects (unless shadowed).

9) Explain instance methods, class methods, and static methods.

Ans: • **Instance Methods:**

The most common type. They require an object to be instantiated, take `self` as their first parameter, and can freely access and modify instance variables and class variables.

• **Class Methods:** They take `cls` (the class itself) as their first parameter instead of `self`. They can modify class state that applies across all instances of the class, but they cannot modify specific object instances.

• **Static Methods:** They take neither `self` nor `cls`. They operate like plain functions that just happen to live inside the class's namespace for logical grouping. They cannot access or modify class or instance states.



10) Why do we use @classmethod and @staticmethod decorators?

Ans: We use these decorators to explicitly tell Python (and other developers) how a method should behave:

- @classmethod: Used when you need to interact with class-level data or when you want to create alternative constructors (eg, creating an object from a string format or a JSON file instead of standard arguments)
- @staticmethod: Used for utility or helper functions that make sense logically to bundle with the class, but do not actually need to know anything about the class or its objects to do their job.